ARMADIN

02.2026

When AI Reverse Engineer Time Travels

About the speaker

Kai Huang

Senior Capability Researcher, Armadin

Kai/Kiwids/ @mhskai2017

Recent discovery (~past 3 months) :

- Three unauth RCE on Solarwinds product line
- Multiple MSRC reports and many other RCEs for Lenovo and Dell.

Works on

Establishing projects that enable the ultimate attacker
Building projects that accelerate researches
Mentoring researchers

Background

Previously Offensive Security Consultant at SpecterOps

Agenda

01 How TTD works

02 Recording and replaying a trace

03 Querying traces programmatically

04 Case studies: LLM with TTD

05 Costs and limits

06 Resources and honorable mentions

Time Travel Debugging Concept

Live debugging

- State exists only while the process runs
- Miss the write, and you restart and reproduce
- Comparing two earlier points means two runs

Time Travel Debugging

- Every thread is recorded into a trace file
- Replay forward or backward, deterministically
- Share the trace and a position, and a teammate lands on the same instruction



Catch it once

Record the failing run one time. Races, timing bugs, and one-shot malware replay the same way every time.



Work backward

Start at the crash or the bad value and step back to the instruction that wrote it.



Query the run

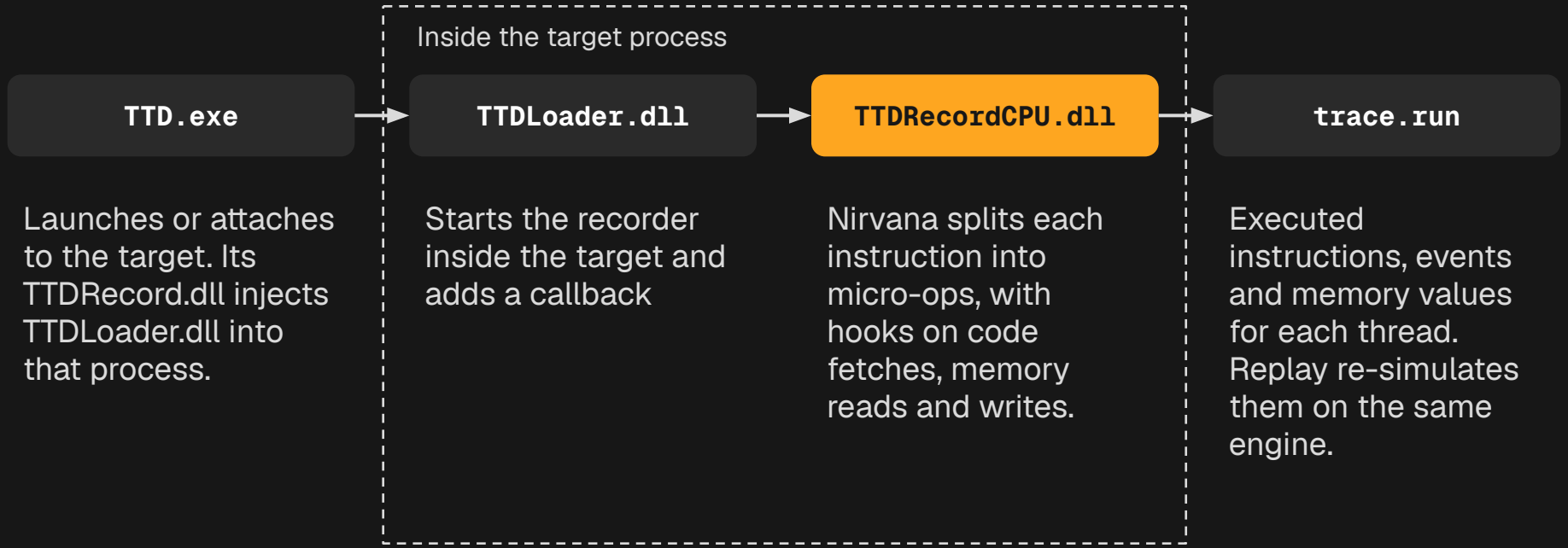
Ask for every call to an API or every write to an address, instead of stepping through code.



Ground an LLM

Hand the trace to an LLM, so its answers come from what executed, not from guesses.

TTD.exe injects a recorder that emulates every instruction the target runs



Three Steps

Record

WinDbg or TTD.exe attaches to a user-mode process and writes the execution of every thread into a .run trace file. Recording needs administrator rights.

Index

When recording stops, TTD builds an .idx index for fast lookups. Expect it to be as large as the trace or larger. Rebuild it any time with !tt.index.

Replay

Open the .run file in WinDbg and step in either direction. History is read-only: you can read memory at any position, but you cannot change it.

How To Time Travel

In WinDbg

1. Run WinDbg as administrator
2. File > Start debugging > Launch executable (advanced)
3. Check Record with Time Travel Debugging
4. Reproduce the behavior, then stop recording

With TTD.exe

Launch and record

```
ttd.exe -out C:\traces target.exe
```

Attach to a running process

```
ttd.exe -out C:\traces -attach <PID>
```

Include child processes

```
ttd.exe -out C:\traces -children target.exe
```

Record every launch of a program

```
ttd.exe -out C:\traces -monitor target.exe
```

Expect a 5x to 20x slowdown while recording. Once TTD attaches it stays until the process exits, so end the target as soon as you have the behavior you need.

Six commands cover most replay navigation

Command	What it does	Reach for it when
`g-`	Runs backward to the previous breakpoint, event, or start of the trace	You want what led to a crash or exception
`p- / t-`	Steps back over, or into, one instruction or line	You overshot inside a function
`!tt 50`	Jumps to the point 50% of the way through the trace	You need to get oriented in a long trace
`!tt <pos>`	Jumps to an exact position, such as 1A2:3F	A teammate sent you a position
`!positions`	Shows the current position of every thread	You want to share where you are
`ba w8 <addr>, g-`	Breaks on a write, then runs backward to it	You need to know who last wrote a value

WinDbg Data Model Support

```
dx @$cursession.TTD.Calls("kernelbase!CreateFileW")
```

Every call, with its parameters and return value

```
dx @$cursession.TTD.Memory(0x7ff000, 0x7ff008, "w")
```

Every write to that address range

```
dx @$curprocess.TTD.Events.Where(e => e.Type == "Exception")
```

Every exception in the trace

```
dx @$curprocess.TTD.Lifetime
```

First and last position in the trace

Programmatic Time Travel

Data model (dx)	Script queries with dx @$cursession.TTD.* or JavaScript extensions. Quick to start, but you still manage a debugger session in the background.
TTD Replay API	Microsoft documents a NuGet SDK that opens and parses traces directly, with no debugger session. Wrappers such as ttd-bindings expose it to Python.
TTDObjectsPy	Wraps the Replay API as MCP tools and Python helpers, so an LLM agent like Codex can seek to positions, read registers and memory, and query calls.

Microsoft's TTD SDK

Replay API

- TTD/ReplayApi
- The same engine WinDbg uses for TTD, now public
- Opens a .run trace with no debugger session
- Built for coverage, memory maps, object lifetime, and data race tools

Live Recorder API

- TTD/LiveRecorderApiSample
- A program records chosen stretches of its own threads
- Adds custom events, heap dumps, and snapshots to the trace
- Recording still needs admin; the sample launches TTD.exe elevated

Both ship in the Microsoft.TimeTravelDebugging.Apis NuGet package. The API is experimental (version 0.x) and may break binary compatibility between TTD releases. Source: Microsoft, WinDbg-Samples TTD SDK.

Examples and Terminologies

Open a trace, seek, read state, run to an event

```
auto [engine, result] = TTD::Replay::MakeReplayEngine();
engine->Initialize(L"trace.run");

TTD::Replay::UniqueCursor cursor(engine->NewCursor());
cursor->SetPosition(engine->GetLifetime().Begin);

auto regs = cursor->GetCrossPlatformContext();
auto mem  = cursor->QueryMemoryRange(address);

cursor->SetEventMask(EventMask::MemoryWatchpoint
                   | EventMask::Exception);
cursor->ExecuteForward();
```

Engine

Loads one .run trace and exposes its timeline, threads, and modules.

Cursor

Seeks and steps either way, then reads registers and memory.

Position

128-bit ID of one executed instruction; positions compare numerically.

Index (.idx)

Built on first load, about the trace's size; makes later loads fast.

Source: Microsoft, WinDbg-Samples TTD docs (IReplayEngine.h usage pattern and Concepts).

Case study : LSASS CRASHED

SECTION 04

An LSASS Crash

1 min

until wininit restarts the machine after LSASS dies, taking any live session with it

1 frame

of state in a crash dump: the fault, but none of the writes that caused it

100%

of the run up to the fault stays in the trace, which survives the reboot

Recording LSASS needs a test machine with LSA protection (PPL) and user-mode shadow stacks turned off.

Quick Demo

SECTION 06

TTD Has Real Costs

Slowdown	Recording slows the target 5x to 20x. GUI automation can race the app and capture can fail, so budget time to make recording reliable.
Disk	Traces reach several gigabytes within minutes, and the index adds as much again. Record the shortest window and clean up afterward.
Scope	User mode only: no kernel, no drivers, and no protected (PPL) processes. Antivirus and frameworks such as Electron can break recording.
Sensitivity	A trace holds the memory, file paths, and registry contents the target touched. Handle it like evidence before you share it.

SECTION 06

TTDObjectsPy



Resources

SECTION 06

Start with Microsoft's documentation and FLARE's free intro module

Microsoft documentation

- TTD overview
- TTD.exe command-line recorder
- Replay a trace
- TTD object model
- TTD Replay API (NuGet SDK)
- WinDbg-Samples: TTD
- TTD release notes

Guides and courses

- Mandiant FLARE: An Introduction to TTD
- Google Cloud: TTD and .NET process hollowing
- TTD Lab: slides and hands-on labs
- Next-level reversing: Binary Ninja + TTD
- LRQA: debugging shellcode with TTD
- IDA Pro: WinDbg TTD debugging
- Invoke RE: Advanced Malware Binary Triage

Resource lists on slides 20–23 build on awesome-ttd by xusheng6, plus Microsoft, SpecterOps, and vendor sources.

Open-source tools that build on the TTD engine

Tool	Built on	What it adds
TTDObjectsPy	TTD Replay API	MCP tools and Python for LLM agents
ttd-bindings	TTD Replay API	Python bindings for trace scripting
ttddbg	IDA Pro	Replays TTD traces inside IDA
WindbgTTD-TrackReg	WinDbg extension	Finds where a value came from
ttd-capa	capa	Capabilities from traced API calls
WinDbg-Libraries	Debugger data model	C++ layer for WinDbg extensions
Binary Ninja TTD adapter	DbgEng	Binary Ninja's TTD integration

Write-ups and talks that show TTD on real targets

Write-ups

- SpecterOps: Time Travel Debugging with Codex
- Binary Ninja: Flare-On 11 TTD challenge
- Huntress CTF 2025: Rust Tickler 1–3
- JLORAT: a Rust RAT under TTD

Talks and videos

- Defrag Tools #185: TTD introduction
- Defrag Tools #186: TTD advanced
- CppCon 2017: TTD in WinDbg
- Binary Ninja: working with TTD
- Flare-On 2025 challenge 7 walkthrough

Outside Windows user mode, other record-and-replay tools fill the gaps

Tool	Platform and scope	Pricing
rr	Linux user mode, with a GDB front end	Free, open source
GDB reverse execution	Many targets, through process record	Free, open source
REnigma	Full-system recording for security research	Commercial

PANDA records a whole virtual machine, then runs your analysis during replay

How it records

- panda-re.mit.edu
- Built on QEMU, so the kernel, drivers, and every guest process land in one recording
- Stores a starting snapshot plus only the nondeterministic inputs: interrupts, DMA, user input
- A 9-billion-instruction FreeBSD boot fits in a few hundred MB

How you analyze it

- github.com/panda-re/panda
- Plugins hook 40+ emulator callbacks, written in C, C++, or Python (PyPANDA)
- Stock plugins cover taint tracking, system calls, and OS introspection
- Slow analysis runs at replay time, so it cannot change what the guest did

Intel PinPlay records one process as a pinball and replays it under any Pintool

How it records

- Program Record/Replay Toolkit
- Built on the Pin instrumentation engine and shipped inside Intel SDE
- A pinball holds the starting memory and registers, plus the result of every system call
- Records on Windows, Linux, or macOS; capture runs 100x to 200x slower

How you analyze it

- DrDebug command-line usage
- Any PinPlay-enabled Pintool replays the pinball with the same addresses and thread order every time
- DrDebug attaches GDB to the replay for debugging multithreaded bugs
- A long pinball can be sliced into small regional pinballs

Source: Intel, Using PinPlay for Reproducible Analysis and Replay Debugging, PLDI 2016 tutorial (capture overhead).

TTD is the practical pick on Windows; PANDA and PinPlay cover what it cannot

Tool	TTD	PANDA	Intel PinPlay
Scope	One Windows user-mode process, plus children	Whole QEMU VM: kernel, drivers, every process	One x86 process on Windows, Linux, or macOS
What it stores	Every thread's execution, plus an index	Start snapshot and nondeterministic inputs	Start state and system call results
Recording cost	5x to 20x slower; GBs in minutes	Runs under emulation; compact logs	100x to 200x slower
Analysis	Step both ways in WinDbg; query with dx	Plugins in C, C++, or Python at replay	Pintools at replay; GDB via DrDebug
Best for	Windows malware and app bugs	Kernel, firmware, cross-process taint	Multithreaded bugs, repeatable Pintools
Price	Free with WinDbg	Free, open source	Free, Intel license

Honorable mentions: Undo and eShard

Undo

- undo.io
- UDB debugger and LiveRecorder for Linux on x86-64 and ARM64
- C, C++, Go, Rust, and Java
- Records failing CI and test runs, then replays them in VS Code or IntelliJ

eShard

- esReverse and Epoch
- Full-system TTD: kernel, drivers, and every userland process in one trace
- Windows, Linux, Android, and IoT targets
- Epoch (Sept 2026) adds a local MCP server that can navigate traces